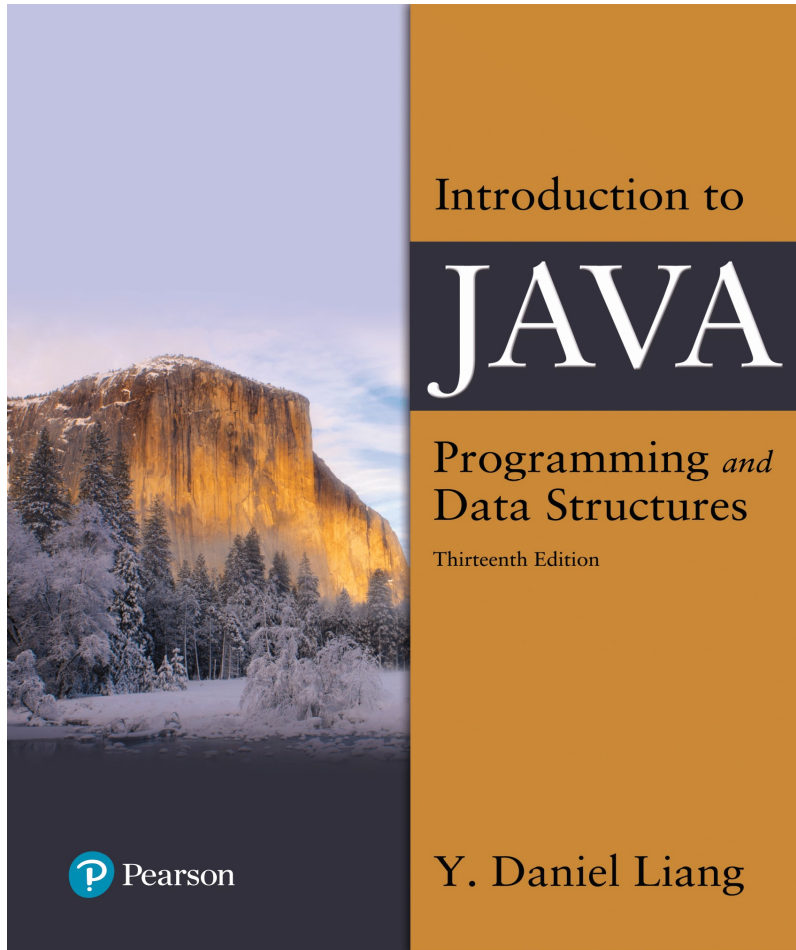


Introduction to Java Programming and Data Structures

Thirteenth Edition

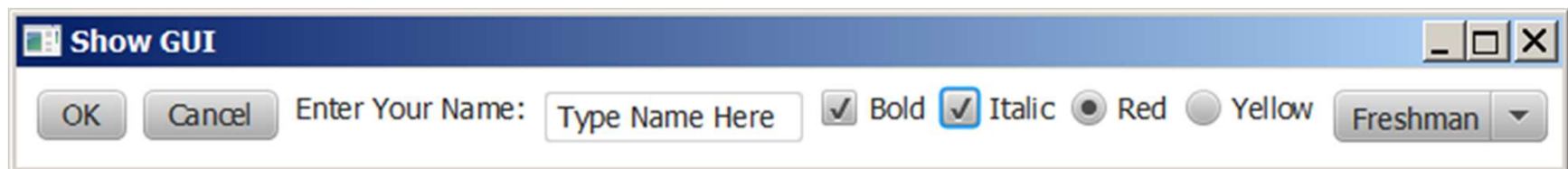


Chapter 9

Objects and Classes

Motivations

After learning the preceding chapters, you are capable of solving many programming problems using selections, loops, methods, and arrays. However, these Java features are not sufficient for developing graphical user interfaces and large scale software systems. Suppose you want to develop a graphical user interface as shown below. How do you program it?



Objectives (1 of 2)

9.1 To describe objects and classes, and use classes to model objects (§ 9.2).

9.2 To use UML graphical notation to describe classes and objects (§ 9.2).

9.3 To demonstrate how to define classes and create objects (§ 9.3).

9.4 To create objects using constructors (§ 9.4).

9.5 To define a reference variable using a reference type and access objects via object reference variables (§ 9.5).

9.6 To access an object's data and methods using the object member access operator (.) (§ 9.5.1).

9.7 To define data fields of reference types and assign default values for an object's data fields (§ 9.5.2).

9.8 To distinguish between object reference variables and primitive data type variables (§ 9.5.3).

Objectives (2 of 2)

- 9.9** To use the Java library classes **Date**, **Random**, and **Point2D** (§ 9.6).
- 9.10** To distinguish between instance and static variables and methods (§ 9.7).
- 9.11** To define private data fields with appropriate **get** and **set** methods (§ 9.8).
- 9.12** To encapsulate data fields to make classes easy to maintain (§ 9.9).
- 9.13** To develop methods with object arguments and differentiate between primitive-type arguments and object-type arguments (§ 9.10).
- 9.14** To store and process objects in arrays (§ 9.11).
- 9.15** To create immutable objects from immutable classes to protect the contents of objects (§ 9.12).
- 9.16** To determine the scope of variables in the context of a class (§ 9.13).
- 9.17** To use the keyword **this** to refer to the calling object itself (§ 9.14).

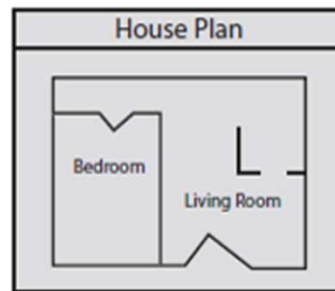
Classes

- In an object-oriented programming language, like Java, you create programs that are made of objects.
- In software, an object has two capabilities:
 - An object can store data.
 - An object can perform operations.
- The data stored in an object are commonly called *attributes* or *fields*.
- The operations that an object can perform are called *methods*.

Classes and Instances

- Many objects can be created from a class.
- Each object is independent of the others.

Blueprint that describes a house



Instances of the house described by the blueprint



Objects and Programs

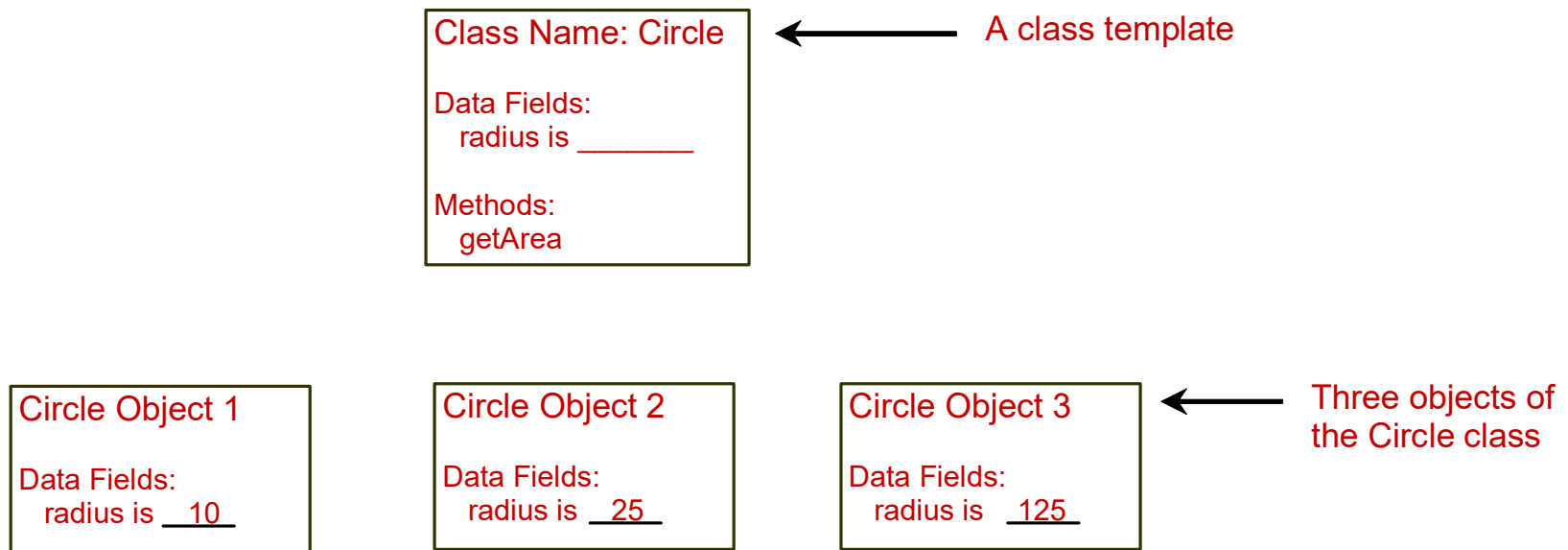
- For example, the String class provides methods:
 - Examples: `length()` and `charAt()` methods

```
String greeting = new String("Hello World");  
int len = greeting.length();  
char c1 = greeting.charAt(0);
```

The *greeting* variable
Holds the address of
a String object.



Objects



An object has both a state and behavior. The state defines the object, and the behavior defines what the object does.

Classes (2 of 2)

```
class Circle {  
    /** The radius of this circle */  
    double radius = 1.0;  
  
    /** Construct a circle object */  
    Circle() {  
    }  
  
    /** Construct a circle object */  
    Circle(double newRadius) {  
        radius = newRadius;  
    }  
  
    /** Return the area of this circle */  
    double getArea() {  
        return radius * radius * 3.14159;  
    }  
}
```

← Data field

← Constructors

← Method

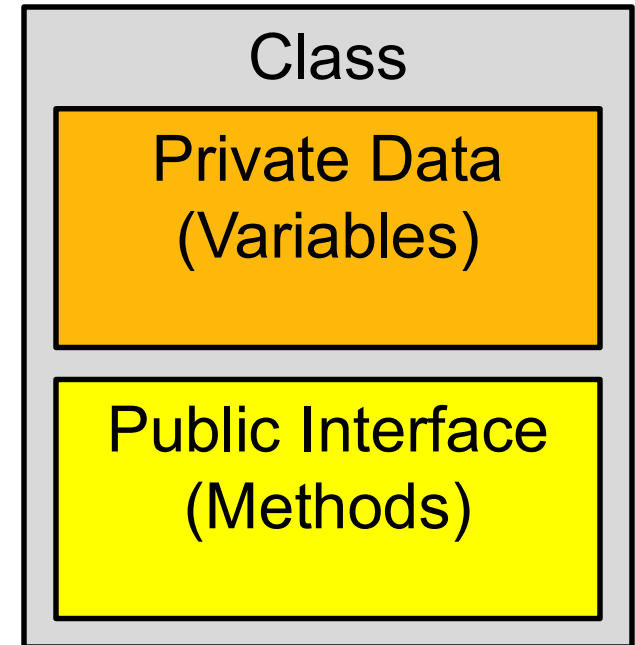
Diagram of a Class

❑ Private Data

- Each object has its own private data that other objects cannot directly access
- Methods of the public interface provide access to private data, while hiding implementation details:
 - This is called Encapsulation

❑ Public Interface

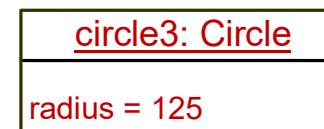
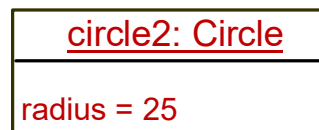
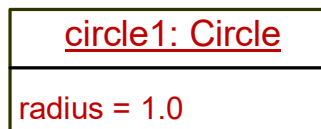
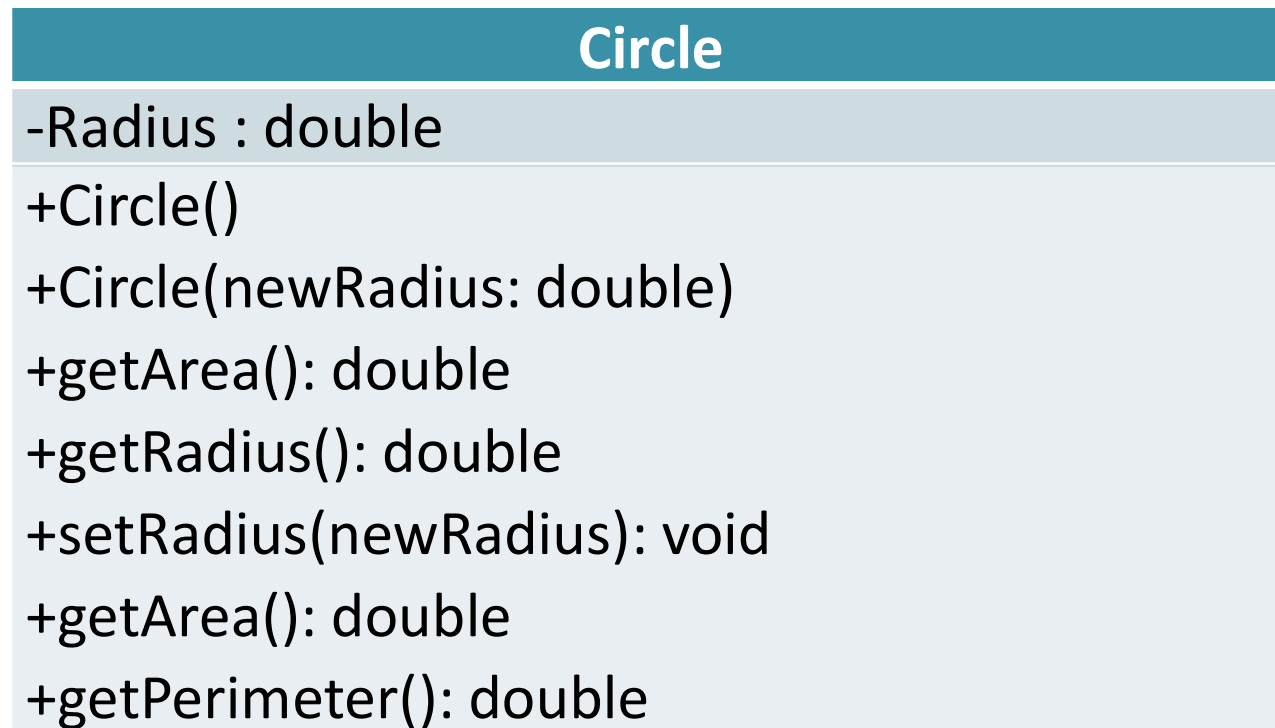
- Each object has a set of methods available for other objects to use



An access modifier is a Java key word that indicates how a field or method can be accessed.

Modifier on members in a class	Accessed from the same class	Accessed from the same package	Accessed from a subclass	Accessed from a different package
public	✓	✓	✓	✓
protected	✓	✓	✓	–
default	✓	✓	–	–
private	✓	–	–	–

UML Class Diagram



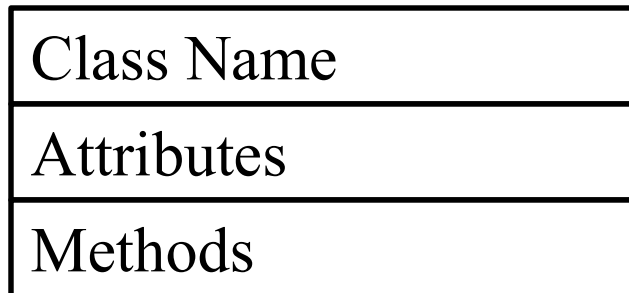
← UML notation
for objects

Designing a Class

- When designing a class, decisions about the following must be made.
 - what data must be accounted for
 - what actions need to be performed
 - what data can be modified
 - what data needs to be accessible
 - any rules as to how data should be modified
- Class design typically is done with the aid of a Unified Modeling Language (UML) diagram.

UML Class Diagram

- A UML class diagram is a graphical tool that can aid in the design of a class.
- The diagram has three main sections.



UML diagrams are easily converted to Java class files. There will be more about UML diagrams a little later.

- The class name should concisely reflect what the class represents.

Attributes

- The data elements of a class define the object to be instantiated from the class.
- The attributes must be specific to the class and define it completely.
- Example: A rectangle is defined by
 - length
 - width
- The attributes are then accessed by methods within the class.

Methods

- ❑ The class' methods define the actions that an instance of the class can perform

- ❑ Methods headers have a format:

```
AccessModifier ReturnType  
MethodName (Parameters)  
  
{  
    // Method body.  
}
```

- ❑ Methods that need to be used by other classes should be made public.

Methods

- ❑ The attributes of a class might need to be:
 - changed
 - accessed
 - calculated
- ❑ The methods that change and access attributes are called *accessors* and *mutators*.

Stale Data

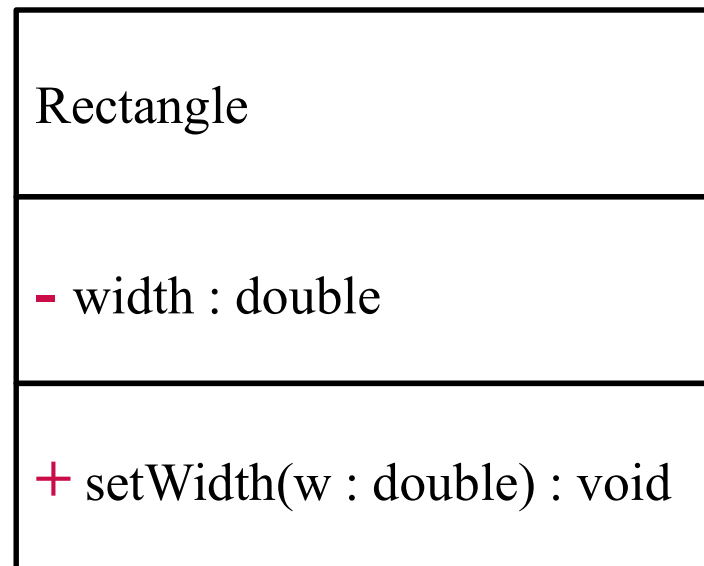
- ❑ Rather than use an `area` variable in a rectangle class:

```
public double getArea()  
{  
    return length * width;  
}
```

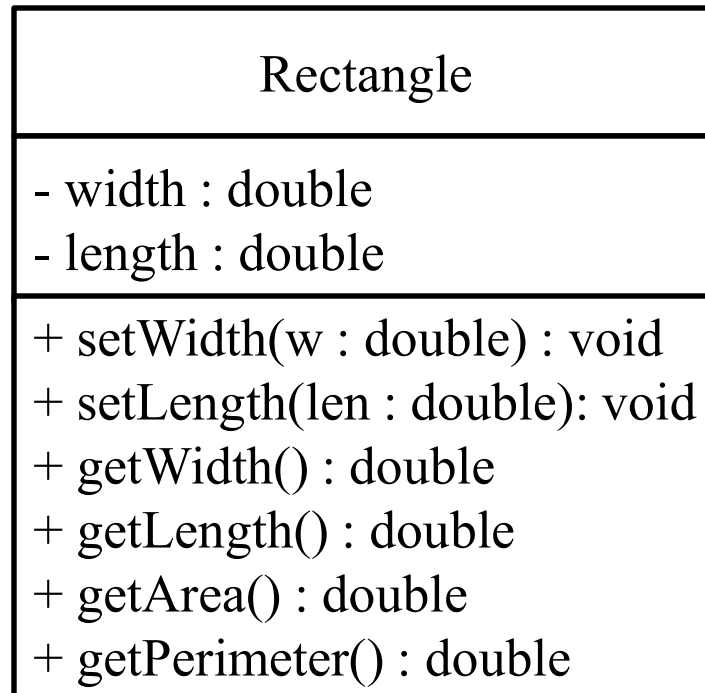
- ❑ This dynamically calculates the value of the rectangle's area when the method is called.
- ❑ Now, any change to the `length` or `width` variables will not leave the area of the rectangle stale.

UML Data Type and Parameter Notation

- ❑ UML diagrams are language independent.
- ❑ UML diagrams use an independent notation to show return types, access modifiers, etc.



The UML Diagram for Rectangle class



A Driver Program

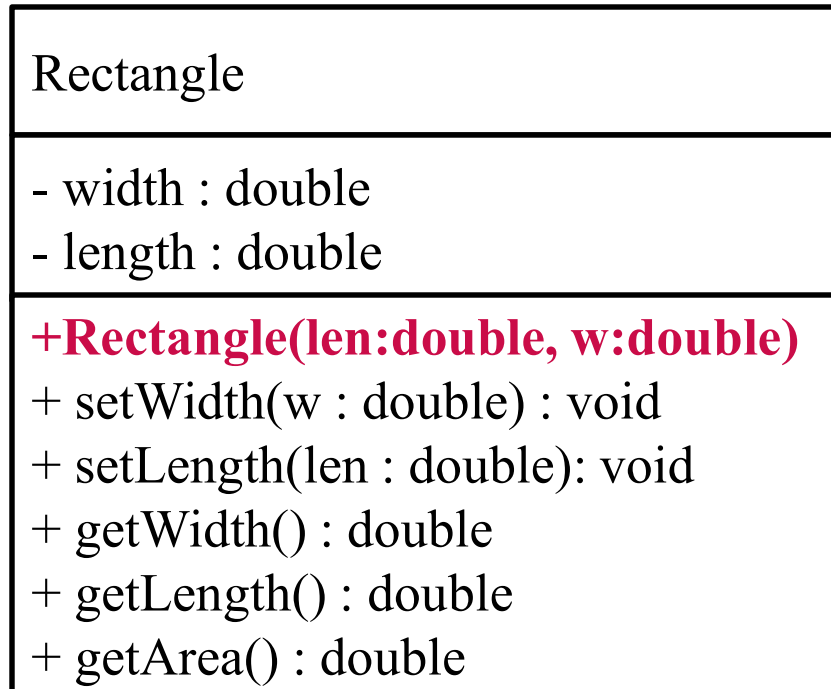
- ❑ An application in Java is a collection of classes that interact.
- ❑ The class that starts the application must have a `main` method.
- ❑ This class can be used as a *driver* to test the capabilities of other classes.
- ❑ In the `Rectangle` class example, notice that there was no `main` method.

Constructors

- ❑ Classes can have special methods called *constructors*.
- ❑ Constructors are used to perform operations at the time an object is created.
- ❑ Constructors typically initialize instance fields and perform other object initialization tasks.

Constructors in UML

- ❑ In UML, the most common way constructors are defined is:



The Random Class

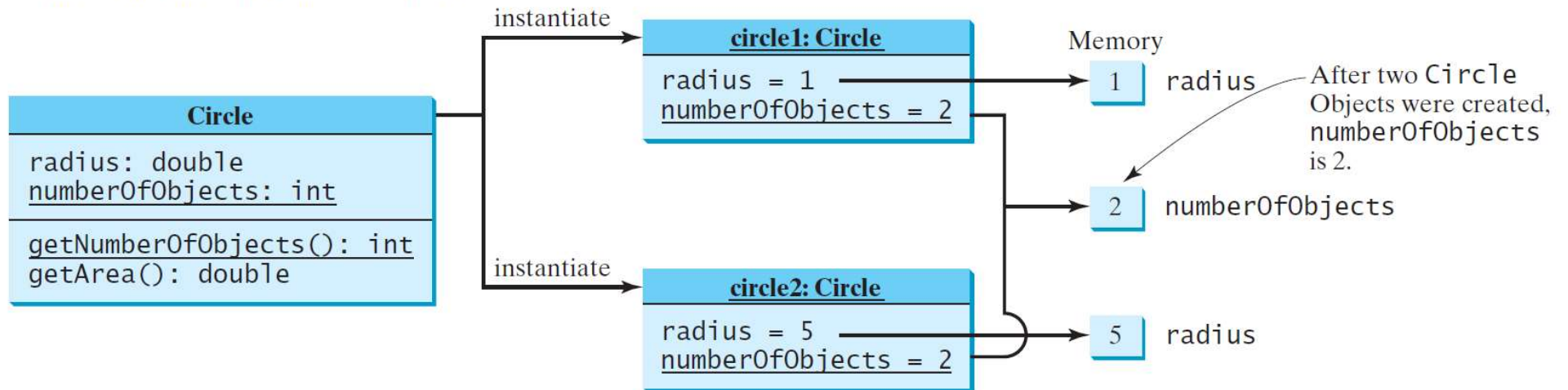
You have used Math.random() to obtain a random double value between 0.0 and 1.0 (excluding 1.0). A more useful random number generator is provided in the java.util.Random class.

java.util.Random	
+Random()	Constructs a Random object with the current time as its seed.
+Random(seed: long)	Constructs a Random object with a specified seed.
+nextInt(): int	Returns a random int value.
+nextInt(n: int): int	Returns a random int value between 0 and n (exclusive).
+nextLong(): long	Returns a random long value.
+nextDouble(): double	Returns a random double value between 0.0 and 1.0 (exclusive).
+nextFloat(): float	Returns a random float value between 0.0F and 1.0F (exclusive).
+nextBoolean(): boolean	Returns a random boolean value.

Static Variables, Constants, and Methods

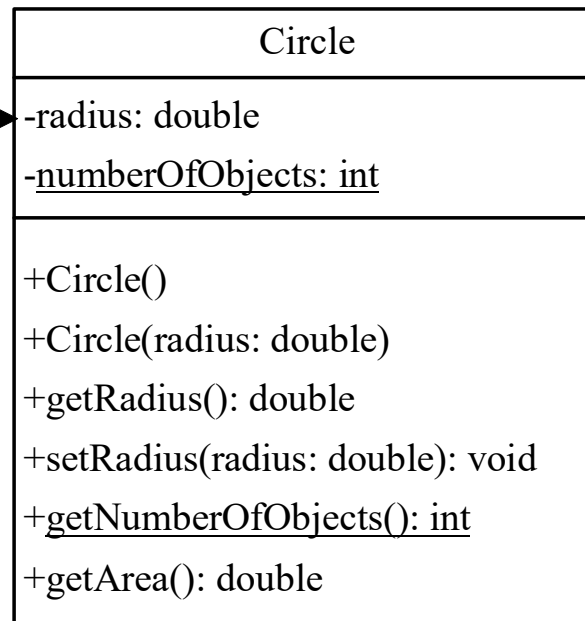
UML Notation:

underline: static variables or methods



Example of Data Field Encapsulation

The - sign indicates
private modifier



The radius of this circle (default: 1.0).

The number of circle objects created.

Constructs a default circle object.

Constructs a circle object with the specified radius.

Returns the radius of this circle.

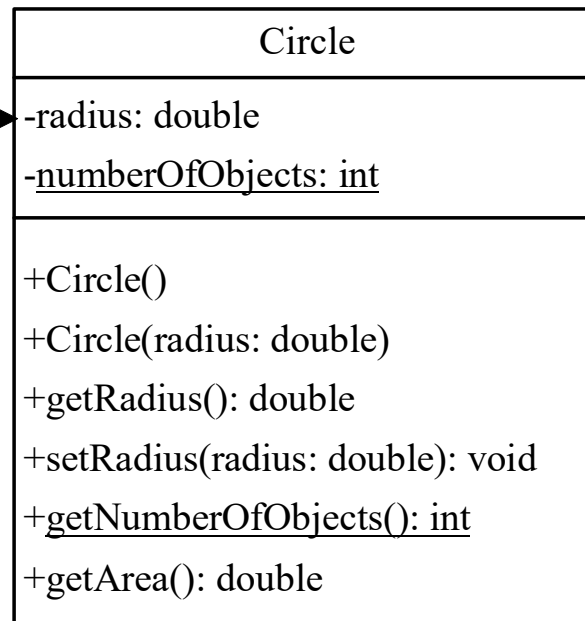
Sets a new radius for this circle.

Returns the number of circle objects created.

Returns the area of this circle.

Example of Data Field Encapsulation

The - sign indicates
private modifier



The radius of this circle (default: 1.0).

The number of circle objects created.

Constructs a default circle object.

Constructs a circle object with the specified radius.

Returns the radius of this circle.

Sets a new radius for this circle.

Returns the number of circle objects created.

Returns the area of this circle.

Circle

TestCircle

The this Keyword

- ❑ The this keyword is the name of a reference that refers to an object itself. One common use of the this keyword is reference a class's *hidden data fields*.
- ❑ Another common use of the this keyword to enable a constructor to invoke another constructor of the same class.

Reference the Hidden Data Fields

```
public class F {  
    private int i = 5;  
    private static double k = 0;  
  
    void setI(int i) {  
        this.i = i;  
    }  
  
    static void setK(double k) {  
        F.k = k;  
    }  
}
```

Suppose that f1 and f2 are two objects of F.
F f1 = new F(); F f2 = new F();

Invoking f1.setI(10) is to execute
 this.i = 10, where **this** refers f1

Invoking f2.setI(45) is to execute
 this.i = 45, where **this** refers f2

Scope of Variables

- ❑ The scope of instance and static variables is the entire class. They can be declared anywhere inside a class.
- ❑ The scope of a local variable starts from its declaration and continues to the end of the block that contains the variable. A local variable must be initialized explicitly before it can be used.

Calling Overloaded Constructor

```
public class Circle {  
    private double radius;
```

```
    public Circle(double radius) {  
        this.radius = radius;  
    }
```

→ this must be explicitly used to reference the data field radius of the object being constructed

```
    public Circle() {  
        this(1.0);  
    }
```

→ this is used to invoke another constructor

```
    public double getArea() {  
        return this.radius * this.radius * Math.PI;  
    }  
}
```

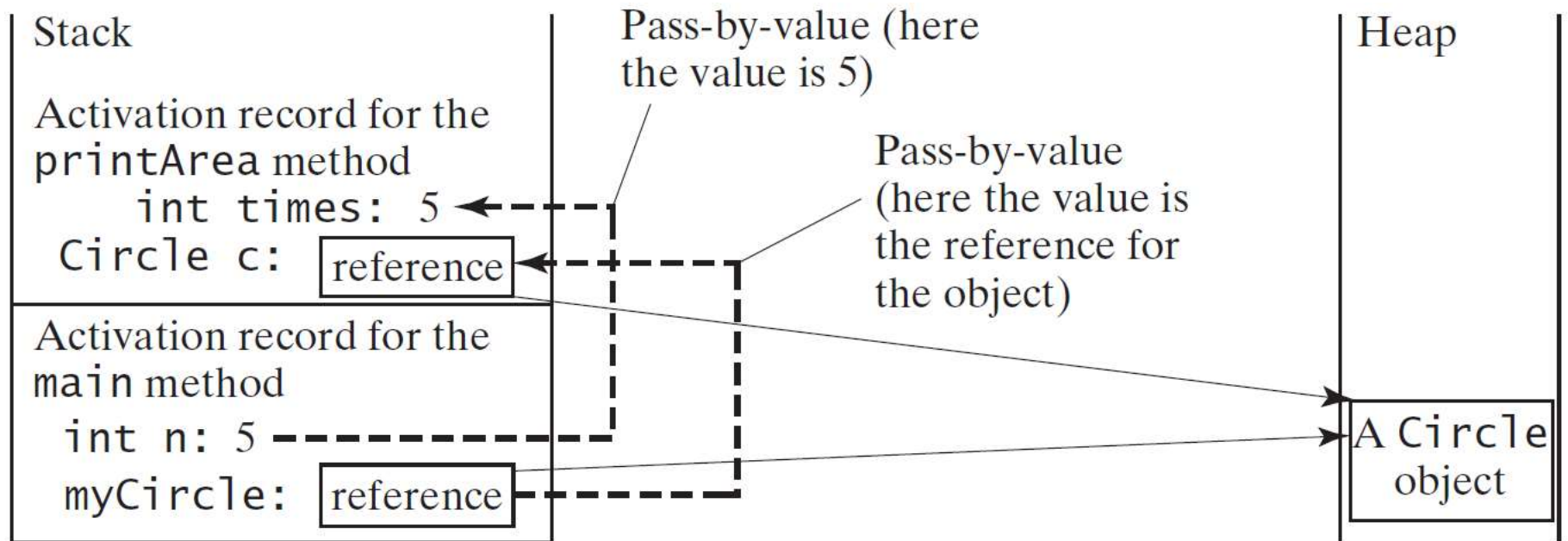
↓
Every instance variable belongs to an instance represented by this, which is normally omitted

Passing Objects to Methods

- ❑ Passing by value for primitive type value (the value is passed to the parameter)
- ❑ Passing by value for reference type value (the value is the reference to the object)

TestPassObject

Passing Objects to Methods, cont.



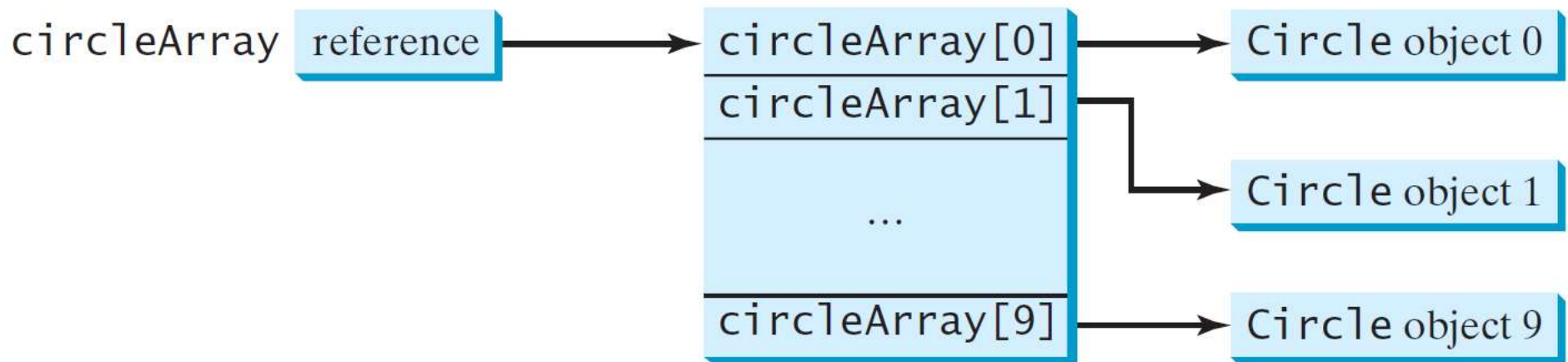
Array of Objects

```
Circle[] circleArray = new Circle[10];
```

An array of objects is actually an *array of reference variables*. So invoking `circleArray[1].getArea()` involves two levels of referencing as shown in the next figure. `circleArray` references to the entire array. `circleArray[1]` references to a `Circle` object.

Array of Objects, cont.

```
Circle[] circleArray = new Circle[10];
```



Array of Objects, cont.

Summarizing the areas of the circles

- ❑ create an array of five random circles.
- ❑ Print the circles and their area.
- ❑ Print their total area.

TotalArea

Objects & Classes

Rational Class



Rational Numbers UML

Rational
- numerator : int - denominator : int
+ Rational() + Rational(n : int, d : int) + getNumerator() : int + setNumerator(n : int) + getDenominator() : int + setDenominator(d : int) + add(r : Rational) : Rational + subtract(r : Rational) : Rational + divide(r : Rational) : Rational + multiply(r : Rational) : Rational + equals(r : Rational) : Boolean + compareTo(r : Rational) : int + toString() : String - gcd(n : int , d : int) : int

default value for a Rational object is 0/1
Rational objects must be in reduced form
Rational objects should not be modified by any of the methods
toString() format is “numerator/denominator”

- Given n , write a program that will compute the following summation series using the Rational class. output should be as follows:

$$\frac{1}{1} + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} = \text{result}$$

- ❑ Given n Rational numbers, find the maximum